

QA & DEBUG REPORT

Web Application Production-Readiness Evaluation • RealWorld (Conduit)

Role Title:	Automation & QA Developer	Target Web App:	Conduit (demo.realworld.io)
Candidate Name:	Abhik	Backend API URL:	api.realworld.io/api
Date of Audit:	June 2, 2026	Assessment Duration:	2 Hours Total (Timebox limit)

1. Executive Summary

A rigorous, comprehensive quality assurance audit was conducted against the **RealWorld (Conduit)** social publishing web platform. The primary objective was to uncover critical functional failures, security vulnerabilities, UX bottlenecks, and architectural issues that restrict production readiness. Testing was conducted across authentic user flows including registration, session management, article creation, tag filtering, and mobile viewports. A total of **6 notable issues** were identified. Among these, the client-side session expiration handling represents a **Critical** usability barrier, and a stored HTML parsing flaw poses a **High** security risk. Resolving these issues is highly recommended prior to launching a production instance.

2. Detailed Bug Report Table

#	Title / Summary	Steps to Reproduce	Expected vs Actual	Severity	Suspected Cause
1	Broken JWT Expiration Handling	1. Log in to the application. 2. Force JWT expiration (or manually edit local storage 'jwt' value to be expired). 3. Click 'Favorite' on any post or try to follow a user.	Expected: App detects expired token, logs user out, and redirects to login page with an alert message. Actual: UI remains in logged-in state. Actions fail silently, throwing unhandled 401 Unauthorized API responses in console.	Critical	App stores JWT token in client state without decoding and validating the 'exp' claim locally, and lacks a global Axios/Fetch response interceptor to handle 401 statuses.
2	Stored XSS Vulnerability in Comments	1. Log in and navigate to an article. 2. In the comments input field, paste: . 3. Post the comment and refresh the page.	Expected: Input tags are sanitized and rendered safely as text literals. Actual: Browser processes and executes the raw HTML script directly on load, enabling cross-site script execution.	High	Frontend comment component renders body using unsafe bindings (like dangerouslySetInnerHTML or v-html) without a DOM sanitizer like DOMPurify.
3	Broken Profile Image Placeholders	1. Register a new user account without upload of a custom profile avatar. 2. Open comment sections or the user profile overview page.	Expected: App displays a standard fallback SVG avatar or beautiful initial-based graphic placeholder. Actual: App attempts to load a hard-coded broken link returning 404, rendering a broken image icon and console errors.	Medium	The fallback image property is bound to an expired external domain asset (static.productionready.io/images/smiley-cyrus.png) which is no longer hosted.
4	Lack of Password Recovery Page	1. Navigate to the login view (/login). 2. Review the layout for a 'Forgot Password?' or account recovery link.	Expected: A secure recovery link initiating an email-based password reset workflow. Actual: No forgot-password mechanism exists. Users are locked out permanently upon forgetting passwords, blocking production use.	High	Authentication frontend routes and layouts completely omit recovery forms, and the backend lacks a password reset endpoint and mailer integration.
5	Missing Visual Loading State on Tag Filter	1. Open the home page. 2. Click a tag under the 'Popular Tags' list (simulate slow connection). 3. Watch the article list area.	Expected: Loading indicator or skeleton cards show immediately to confirm the filter request is in progress. Actual: Screen remains completely static. App appears locked or unresponsive until data finishes fetching several seconds later.	Medium	API request calls do not trigger a 'loading' boolean flag in the state container, meaning UI does not conditionally render a spinner during fetching.
6	Overlapping Header Menu on Mobile view	1. Access application on a screen width below 480px (e.g. mobile viewport). 2. Observe the top navigation header structure.	Expected: Clean responsive menu structure (such as a hamburger drawer) that rescales cleanly. Actual: Navigation links wrap onto multiple lines, overlapping awkwardly with the layout banner and making touch targets extremely tight.	Low	CSS stylesheet lacks responsive media queries for screens under 768px, and has no mobile drawer/hamburger nav component.

3. Root-Cause Analysis (Issue 1: Broken JWT Expiration Handling)

During testing, a critical security and UX vulnerability was observed: when a user's JSON Web Token (JWT) expires, the client interface remains statically in a logged-in state without initiating session termination. The root cause lies in the application's auth state management. The app persists the token in local storage and binds its authentication status to the presence of this token string, completely neglecting to decode and inspect the expiration claim (exp) on the client side. Consequently, when the token expires on the server, the client continues to display authorized UI states. When the user attempts to trigger actions like liking posts or following creators, the backend database correctly rejects the request and returns an unhandled HTTP 401 Unauthorized status. Because the frontend app does not implement global HTTP interceptors to catch these responses, the errors fail silently, leaving the user stranded with no feedback. To resolve this, a robust solution must be implemented at the global HTTP client layer (such as an Axios response interceptor) that automatically traps 401 exceptions, dispatches a logout action to wipe expired credentials, and gracefully redirects the browser back to the login view while alerting the user. Below is the proposed fix to be integrated into the HTTP networking configuration:

```
// src/api/client.js - Global HTTP Response Interceptor Fix
import axios from 'axios';
import store from '../store';
import router from '../router';

const apiClient = axios.create({ baseURL: 'https://api.realworld.io/api' });

// Intercept response to handle global authentication failures
apiClient.interceptors.response.use(
  (response) => response, // Pass successful responses straight through
  (error) => {
    if (error.response && error.response.status === 401) {
      // Token expired or invalid: force client state cleanup
      store.dispatch('auth/logout');
      router.push('/login');
      alert('Your session has expired. Please log in again.');    }
    return Promise.reject(error);
  }
);
```